ICS Network Traffic Analysis with Malcolm

A Hands-On Investigation of the 4SICS Geek Lounge Dataset

Environment Setup → Three-Day Forensic Analysis → Findings

Self-directed learning project · Tools: Malcolm (Zeek + Suricata + Arkime + OpenSearch)

Project type	Self-directed learning / skills demonstration
Toolset	Malcolm on Ubuntu (VMware Workstation VM)
Dataset	4SICS Geek Lounge ICS captures (Netresec), Oct 2015
Files analyzed	151020 (25 MB), 151021 (134 MB), 151022 (200 MB)
Classification	Public training data — not a live incident

Contents

1. Executive Summary

This report documents a self-directed project to learn Malcolm — the open-source network traffic analysis suite developed by CISA — by using it to investigate a well-known public dataset of industrial control system (ICS) traffic. The dataset comprises three packet captures recorded on consecutive days at the 4SICS (now CS3Sthlm) security conference "Geek Lounge," a hands-on lab of real industrial equipment that conference attendees were invited to probe and attack.

Across the three captures, a clear attack progression emerged. Day 1 (Tuesday) was a quiet baseline of normal industrial operations with no attacker activity. Day 2 (Wednesday) showed coordinated reconnaissance from a "client" subnet (192.168.2.0/24): web-interface scanning and — verified by source filtering — device fingerprinting over Modbus and S7comm and port-scanning over DNP3, but no write/control commands. Day 3 (Thursday) marked the escalation to active manipulation: host 192.168.2.166 conducted a sustained Modbus attack (fingerprinting, function enumeration, protocol fuzzing, and writes) against three control devices, with roughly 10,584 coil-write commands acknowledged. Crucially, careful source attribution established that the Siemens PLC was NOT successfully written to by any attacker — correcting an initial impression formed from unfiltered totals.

Bottom line: the dataset captures a textbook ICS kill-chain — reconnaissance escalating to control-layer manipulation — and the investigation demonstrates both the Malcolm toolset and the analytical discipline of separating observation from conclusion and verifying source attribution before attributing activity to an attacker.

2. Objectives & Scope

2.1 Objectives

- Learn to deploy and operate Malcolm from scratch with no prior experience.
- Develop practical network-forensics skills: navigation, filtering, query writing, packet inspection, and alert triage.
- Understand core ICS protocols (Modbus, S7comm, DNP3) well enough to interpret real traffic.
- Produce professional, evidence-backed analysis with explicit confidence levels.

2.2 Scope

In scope: the three 4SICS Geek Lounge captures, analyzed within Malcolm. Out of scope: any live or production environment. The "attackers" are authorized conference participants attacking demonstration equipment; therefore findings are framed as "attack-pattern traffic in a training environment," and statements about severity describe what the equivalent activity would mean in a real plant.

3. Environment Setup

3.1 Platform

Malcolm was run on an Ubuntu 64-bit virtual machine hosted in VMware Workstation. Malcolm is a containerized suite, so Docker provides the runtime; the suite bundles several tools that each process uploaded captures in parallel.

3.2 Components of Malcolm

Component	Role in plain terms
Zeek	Translates raw packets into detailed, human-readable protocol logs (conn, dns, modbus, s7comm, dnp3, etc.).
Suricata	Signature-based detection engine that raises alerts on known suspicious patterns.
Arkime	Full session/packet retrieval — lets you inspect every individual conversation and its raw bytes.
OpenSearch + Dashboards	Search engine and the dashboards that summarize and visualize all the parsed data.
Logstash	The pipeline that ingests and enriches the logs before they reach OpenSearch.

3.3 Bringing Malcolm up

After installation and authentication setup, Malcolm is started from its directory and, once the Logstash pipelines initialize, reports that services are reachable over HTTPS at the VM's address (in this project, https://192.168.171.131/). The startup log confirms the running pipelines — malcolm-input, malcolm-zeek, malcolm-suricata, malcolm-enrichment, malcolm-beats, malcolm-filescan, and malcolm-output — which together represent the full ingest-to-index path. A self-signed certificate warning at first access is expected (the connection is to one's own VM).

3.4 Loading captures

Each PCAP was downloaded via the browser and uploaded through Malcolm's upload page with Zeek and Suricata analysis enabled. Malcolm automatically tags events by source filename (e.g. 4SICSGeekLounge151020), which allowed the three days to coexist in one instance and be isolated or compared later using a simple tag filter. Processing time scaled with file size (minutes for 25 MB; longer for 200 MB).

Practical note: because all captures share one database, every per-day analysis applied a tag filter (e.g. tags: 4SICSGeekLounge151022) to avoid mixing days. A wildcard IP search (192.168.2.*) does not match in the filter “is” operator; the working syntax was a range in the search bar (source.ip: 192.168.2.0 to 192.168.2.255).

4. Methodology

The investigation followed a repeatable analyst workflow, applied to each capture:
Discover → Summarize → Drill in → Verify → Document

- **Discover:** broad Arkime searches across industrial ports to find whether activity of interest exists at all (this catches scans and rejected connections that protocol dashboards omit).
- **Summarize:** protocol-specific dashboards (Modbus, S7comm, DNP3) and the Suricata Alerts dashboard to group and count activity.
- **Drill in:** Arkime sessions to read individual conversations, function codes, and raw detail.
- **Verify:** filter by source IP to confirm attribution; separate "request sent successfully" from "device executed it."
- **Document:** record findings with confidence levels, keeping observations separate from assessments.

Key reading skill — the five fields that matter: source.ip (who), destination.ip (whom), destination.port (which service), network.protocol (which language), and event.action (what they did). Most log interpretation reduces to "Host A asked Host B to do X using protocol Y."
Reference — common ports: 102 = S7comm (Siemens), 502 = Modbus/TCP, 20000 = DNP3, 44818 = EtherNet/IP, 53 = DNS, 80/443 = web admin interfaces.

4.1 Investigation reality — dead-ends and course-corrections

Real analysis is not the clean linear path a summary implies. The following obstacles and corrections shaped the investigation and are recorded deliberately, because working through them is where most of the learning happened:

- **Empty protocol dashboards were misleading.** The ICS/IoT Security Overview returned no results for attacker sources even though attacker ICS traffic clearly existed in Arkime. This led to the realization that protocol dashboards only show fully-parsed conversations — port scans and rejected connections (exactly what attackers generate) are dropped — so a broad Arkime search is required to confirm activity exists at all.
- **Filter syntax failures.** A wildcard IP (source.ip: 192.168.2.*) silently returned nothing in the filter "is" operator. The working approach was a range expression in the search bar (source.ip: 192.168.2.0 to 192.168.2.255). Lesson: when a query returns empty, suspect the syntax before concluding "no data."
- **Mixed-day data required tag discipline.** Because all three captures were loaded into one instance, every per-day view needed a tag filter to avoid silently blending days — and findings had to specify which capture they came from.
- **Tool detours.** An Arkime SPI-view summary path was considered as a fallback when dashboards appeared broken, then abandoned once the dashboard filter syntax was fixed — a reminder to fix the simple cause before switching tools.
- **Browser instability at scale.** Rendering the 600,000-event Day 3 Modbus dashboard exhausted browser memory (Out-of-Memory crash). The fix was to narrow queries before loading, so the page renders only matching records rather than the full dataset.

Why this matters: the obstacles above are not failures but the substance of hands-on work — they are the parts a tool cannot do for you and that a copied tutorial would not contain.

5. Day 1 — Baseline (151020)

Evidence file	4SICS-GeekLounge-151020.pcap (25 MB)
Capture date	20 October 2015 (~2 hours)
Data tag	4SICSGeekLounge151020
Headline	Quiet baseline; normal operations + benign noise; no attackers

5.1 Findings

Finding 1 — Normal industrial control traffic (Siemens S7)

A Siemens S7-1200 PLC communicated over S7comm (port 102) with two workstations; its identity was confirmed at the hardware level by its Siemens MAC OUI. 10.10.10.20 performed Read Variable (0x04) operations (monitoring); 10.10.10.30 performed Write Variable (0x05) operations (legitimate control). Approximately 2,500 reads and 18 writes, all returning Success. **Assessment:** Normal SCADA operation — reads are monitoring, writes are legitimate control from a consistent host on the same control subnet as the PLC.

Finding 2 — Misconfigured device flooding DNS

A Moxa switch (192.168.88.61) issued ~12,331 DNS queries for time.nist.gov, almost all REFUSED/Rejected — a device stuck in a clock-sync retry loop. Benign noise, not malicious.

Finding 3 — Suricata alerts: all benign noise

All 2,481 alerts were a single signature, “SURICATA Ethertype unknown” (rule 2200121, category Generic Protocol Command Decode), firing at regular ~10-second intervals with no source/destination IPs — unrecognized Ethernet frame types from proprietary industrial gear. Benign.

Analysis decision (Day 1)

Faced with 2,481 alerts, the instinct of a beginner is alarm. The decision here was to inspect the alert composition rather than the count — finding all 2,481 were a single benign signature with no source/destination IPs and a clockwork timing pattern (a device heartbeat, not human activity). The verifiable reasoning (single signature + no IPs + regular interval + generic-decode category) is what justified dismissing them as noise rather than guessing.

5.2 Day 1 assessment

No evidence of malicious activity. The value of Day 1 was establishing what normal looks like — essential for recognizing the abnormal on later days — and a key lesson: alert count is meaningless without understanding (2,481 alerts, all noise).

6. Day 2 — Reconnaissance (151021)

Evidence file	4SICS-GeekLounge-151021.pcap (134 MB)
Capture date	21 October 2015
Data tag	4SICSGeekLounge151021

Headline	Coordinated reconnaissance; control layer reached; NO writes

6.1 Alert volume & variety jumped vs. Day 1

Metric	Day 1	Day 2
Total Suricata alerts	2,481	9,202
Distinct alert types	1 (noise)	Many (scans, web attacks)
Attacker source IPs	None	Six 192.168.2.x hosts

6.2 Web-interface scanning (IT layer)

192.168.2.64 (285 alerts) ran Nmap web scans (rule 2024364) against device web interfaces (ports 80/443) of ~10 devices, in an afternoon sweep (~15:42) and a focused evening session (~21:25–21:47). Malformed HTTP request alerts are consistent with scanning. Reconnaissance at the management layer only.

6.3 Control-layer reconnaissance (key finding)

A broad Arkime search across industrial ports returned 489 sessions, confirming attackers reached the control layer. Three hosts, three techniques:

- **192.168.2.42 —** Modbus Read Device Identification (0x2B) on six devices, plus S7comm Read SZL identity queries against the Siemens PLC (192.168.88.30). Read-only fingerprinting; both succeeded.
- **192.168.2.53 —** DNP3 port sweep (20000) across ~14 devices; mostly rejected (scanning, not conversation).
- **192.168.2.88 —** sustained DNP3 connection attempts to a RUGGEDCOM device (192.168.88.95); Zeek flagged abnormal behaviour; no clean DNP3 parsed.

6.4 No control manipulation (verified across all three protocols)

Protocol	Attacker activity	Writes/control?
Modbus (502)	Read Device Identification on 6 devices	None
S7comm (102)	Read SZL identity queries on Siemens PLC	None
DNP3 (20000)	Port scanning + abnormal connections	None (dashboard empty for attackers)

Analysis decision (Day 2)

The pivotal choice on Day 2 was not to stop at the web-scanning finding. Having seen IT-layer scanning, the question raised was specifically “did anyone reach the control layer?” — answered by a deliberate broad Arkime search across industrial ports, which surfaced 489 sessions the alert dashboards alone would not have framed as a control-layer question. Equally deliberate was checking each industrial protocol for write commands to establish, with evidence,

that this was reconnaissance only — a much stronger statement than simply “scanning was observed.”

6.5 Day 2 assessment

A coordinated reconnaissance campaign that progressed from IT-layer web scanning to control-layer device fingerprinting and DNP3 probing — but, verified across Modbus, S7comm, and DNP3, with zero write/operate commands. Discovery only; a significant escalation over Day 1 and a classic precursor to a targeted ICS attack.

7. Day 3 — Active Manipulation (151022)

Evidence file	4SICS-GeekLounge-151022.pcap (200 MB)
Capture date	22 October 2015
Data tag	4SICSGeekLounge151022
Headline	Modbus attack with writes; Siemens PLC not compromised

7.1 Major escalation in ICS activity

Day 3 contained roughly 618,000 ICS-related events — far more than Days 1 or 2 — dominated by Modbus (151,349) with heavy S7comm/COTP. Unlike Day 2, it contained large volumes of Modbus write operations.

7.2 Verified Modbus attack from 192.168.2.166

Filtering Modbus to the single source 192.168.2.166 returned ~99,398 events (all from that host; Malcolm classified it as a Modbus client) against three devices — 192.168.88.60, .61, .95 — combining four behaviours:

- **Device fingerprinting —** Read Device Identification (0x2B), same technique as Day 2.
- **Function enumeration —** systematically walking through Read Coils / Discrete Inputs / Holding & Input Registers against each device.
- **Protocol fuzzing —** invalid/non-standard function codes (unknown-128/160/194/240), which legitimate software never sends.
- **Write commands —** WRITE_MULTIPLE_COILS, WRITE_SINGLE_COIL/REGISTER, WRITE_MULTIPLE_REGISTERS. ~10,584 WRITE_MULTIPLE_COILS were acknowledged (Success); many single writes were rejected. Writes clustered at boundary coil addresses (65533–65535), consistent with fuzzing rather than purposeful set-point control.

Assessment (HIGH confidence, malicious): supported by (a) source on the client subnet that should never write to control devices; (b) ~50% command-rejection rate; (c) accompanying recon/enumeration from the same host; and (d) invalid function codes indicating fuzzing. Successful writes did occur, but the boundary-address/fuzzing pattern points to probing of device behaviour rather than an intent-driven process change.

7.3 Siemens PLC NOT compromised (correction of an initial impression)

The unfiltered Day 3 overview showed S7comm “Write Variable” operations, initially suggesting an attacker wrote to the Siemens PLC. Filtering S7comm strictly to the attacker subnet corrected this: the only attacker-sourced S7comm activity to 192.168.88.30 was four COTP Connection Requests from 192.168.2.133, none of which progressed to any command. The successful S7 writes therefore came from legitimate control stations, not the attackers.
Why this is recorded: it demonstrates that source attribution must be verified before attributing control-layer activity to an attacker; trusting an unfiltered total would have produced a false — and alarming — conclusion.

7.4 Per-protocol verification (attacker source only)

Protocol	Attacker activity	Successful control?
Modbus (502)	Recon + enumeration + fuzzing + writes (192.168.2.166)	Yes — ~10,584 coil-writes acknowledged
S7comm (102)	4 failed connection attempts (192.168.2.133)	No
DNP3 (20000)	Scanning / probing	No

Analysis decision (Day 3)

Day 3 is where judgment mattered most. The unfiltered overview showed S7comm “Write Variable” events, which would have supported a dramatic “attacker wrote to the Siemens PLC” conclusion. The decision was to distrust the aggregate and re-run the view filtered strictly by attacker source — which revealed only four failed connection attempts and reattributed the writes to legitimate control stations. A second judgment was reading intent from the write pattern: writes targeting boundary coil addresses (65533–65535) alongside invalid function codes indicate fuzzing/probing, not purposeful set-point manipulation. Both calls changed the meaning of the findings and neither is something the tool surfaces on its own.

7.5 Day 3 assessment

The escalation from reconnaissance to active manipulation: a sustained Modbus attack with thousands of acknowledged coil-writes against three control devices — the “impact / manipulation of control” phase of the ICS kill-chain — while the Siemens PLC saw only failed connection attempts.

8. The Three-Day Campaign

Day	Capture	Adversary phase	Key result
Day 1	151020	None (baseline)	Normal operations + benign noise; no attackers
Day 2	151021	Reconnaissance / discovery	Web scanning + Modbus/S7comm fingerprinting + DNP3 probing; NO writes
Day 3	151022	Impact / manipulation	Modbus attack by .166 with ~10,584 acknowledged coil-writes; Siemens PLC not compromised

Read together, the captures show a coherent ICS kill-chain: a quiet baseline, a day of mapping and fingerprinting the control systems, and a day of acting on that reconnaissance with write/fuzzing activity against Modbus devices. The same devices recur across days (e.g. the Moxa switch 192.168.88.61 and RUGGEDCOM 192.168.88.95), reinforcing that this was a sustained, methodical campaign rather than isolated noise.

9. Recommendations

Framed as they would apply in a live environment exhibiting this traffic:

- Network segmentation is the single most effective control: the client subnet (192.168.2.0/24) should never be able to reach industrial protocol ports (102/502/20000) on control devices.
- Treat Day 3 as an active control-system compromise: isolate 192.168.2.166 and inspect affected devices (192.168.88.60/.61/.95) for altered coil/register state.
- Detect early-warning recon: alert on Modbus Read Device Identification (0x2B) and S7comm Read SZL from non-engineering hosts — these preceded the attack.
- Detect manipulation: alert on Modbus write functions and on invalid/unknown function codes from unauthorized sources.
- Always verify source attribution before concluding — as the S7comm correction demonstrates.

10. Lessons Learned

- **Verify before you conclude.** Two dramatic first impressions (the S7comm "writes," and "attacker did writes") only held up — or didn't — after filtering by source. This is the most important habit in the field.
- **Separate observation from assessment.** "~10,584 coil-writes acknowledged" is a fact; "malicious, high confidence" is a judgment — stated with reasons and a confidence level.
- **Use the right tool for the question.** Arkime to discover (it shows scans and rejects); protocol dashboards to summarize parsed conversations; back to Arkime to inspect detail.
- **Alert volume ≠ severity.** Day 1's 2,481 alerts were all benign noise; Day 2's smaller, more varied set mattered far more.
- **Dashboards show parsed conversations, not attempts.** An empty protocol dashboard can still hide scanning that only appears in raw sessions.
- **Passive asset identification.** Port + protocol + MAC OUI + traffic direction identifies a device (e.g. a Siemens PLC on 102) without ever querying it.

11. Appendices

Appendix A — ICS port reference

Port	Protocol	Notes
102	S7comm (Siemens)	PLC communication — seen in this dataset
502	Modbus/TCP	Primary attack target on Day 3
20000	DNP3	RUGGEDCOM gear; scanned/probed Day 2–3
44818	EtherNet/IP	Allen-Bradley / Rockwell
80 / 443	HTTP / HTTPS	Device web admin interfaces (Nmap target Day 2)
53	DNS	Source of the Day 1 Moxa flood noise

Appendix B — Key protocol function codes

Protocol	Function	Meaning / concern
S7comm	0x04 Read Variable	Monitoring — low concern

Protocol	Function	Meaning / concern
S7comm	0x05 Write Variable	Changes a PLC value — high concern
S7comm	Read SZL	Identity/diagnostics query — fingerprinting
Modbus	Read Coils / Registers	Monitoring / enumeration
Modbus	Write Single/Multiple Coils/Registers	Changes outputs/values — high concern
Modbus	0x2B Read Device Identification	Vendor/model fingerprinting
DNP3	Read	Monitoring
DNP3	Operate / Direct Operate	Actuation — high concern (none seen from attackers)

Appendix C — Attacker hosts observed

Host	Day	Activity	Layer
192.168.2.64	2	Nmap web scanning (80/443)	IT
192.168.2.42	2	Modbus + S7comm fingerprinting	OT
192.168.2.53	2	DNP3 port sweep	OT
192.168.2.88	2	DNP3 connection attempts	OT
192.168.2.166	3	Modbus attack: recon + fuzzing + writes	OT
192.168.2.133	3	Failed S7comm connection attempts to PLC	OT

Appendix D — Useful queries

Arkime — any attacker traffic on industrial ports:

```
ip == 192.168.2.0/24 && (port == 102 || port == 502 || port == 20000 || port == 44818)
```

Dashboard — isolate one capture and the attacker subnet:

```
tags: 4SICSGeekLounge151022   AND   source.ip: 192.168.2.0 to 192.168.2.255
```

Dashboard — confirmed coil-writes from the attacker:

```
source.ip: 192.168.2.166 and event.action: WRITE_MULTIPLE_COILS and event.result: Success
```

Disclaimer: Analysis performed on a public training dataset (4SICS Geek Lounge, Netresec). The “attackers” were authorized conference participants attacking demonstration equipment; this is not a real-world incident. Findings, confidence levels, and source-verified attribution are stated throughout, with assessments distinguished from raw observations.